PROGRAMMING & CS FUNDAMENTALS

Trees
Visual Guide

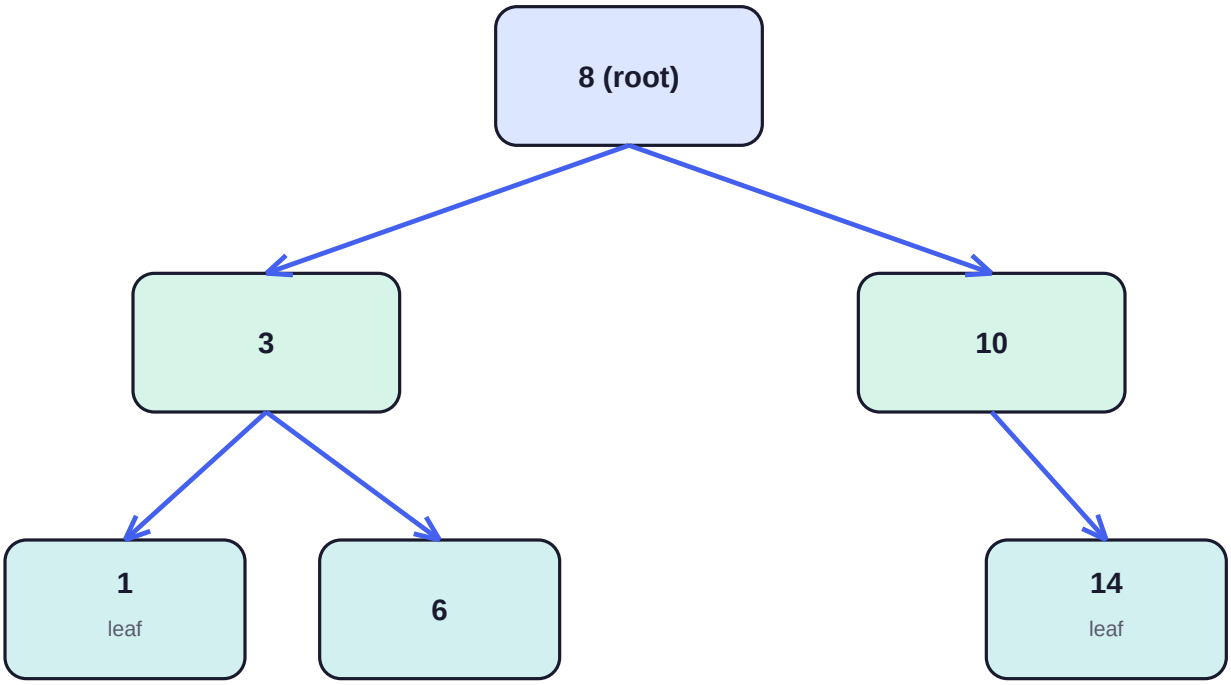
Hierarchies with O(log n) superpowers — when balance is kept.

ANIMATED WALKTHROUGH
Step through pages to play the animation

STEP 1/6

Anatomy of a Tree

Root at top, leaves at bottom, one path between any two nodes.



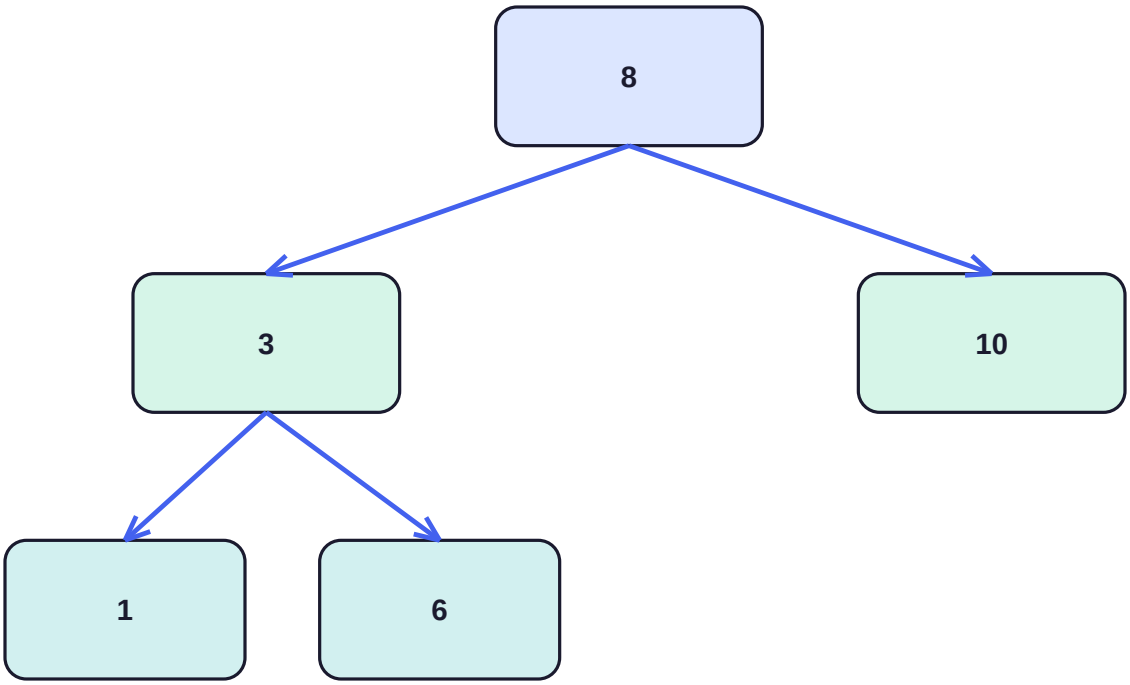
depth(root)=0; height = longest root->leaf path (here: 2). Binary tree: at most 2 children per node.

- Parent/child/sibling, subtree, ancestor — the family vocabulary of hierarchies.
- A tree with n nodes has exactly n-1 edges — one path between any pair.
- File systems, DOM, org charts, decision trees: trees model nesting everywhere.

STEP 2/6

BST: Ordered by Construction

Smaller keys left, larger keys right — search prunes half the tree per step.




```
search(6): 8 -> go left (6<8) -> 3 -> go right (6>3) -> 6 FOUND
insert(4): 8 -> 3 -> 6 -> left of 6 (4<6): attach.  O(h)
delete: 0/1 child -> splice; 2 children -> swap with successor
```

h = height. Balanced: h ~ log n -> O(log n). Degenerate: h = n -> O(n). Balance is EVERYTHING (step 4).

STEP 3/6

Traversals: Three Orders

Same tree, three visit sequences — recursion does the walking.

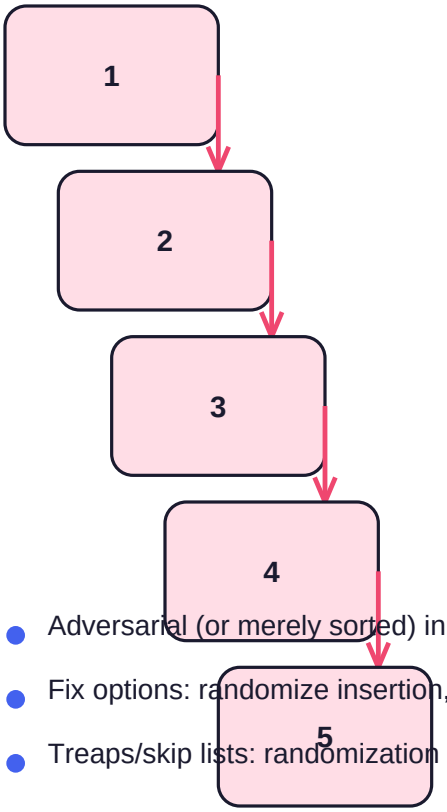
```
inorder   (L, root, R): 1, 3, 6, 8, 10  # sorted!
preorder  (root, L, R): 8, 3, 1, 6, 10  # copy structure
postorder (L, R, root): 1, 6, 3, 10, 8  # delete safely
```

- Inorder on a BST yields sorted order — the cheapest sort you'll ever write.
- Preorder records structure (serialize); postorder children-before-parent (free memory).
- Level order (BFS) visits by depth — needs a queue, not recursion.
- Morris traversal: O(1) space inorder via temporary threading — interview gold.

STEP 4/6

The Degenerate Trap

Insert sorted data into a naive BST and it becomes a linked list.



height = n-1

search = O(n) -- BST advantage: GONE

- Adversarial (or merely sorted) input is common: timestamps, IDs, logs.
- Fix options: randomize insertion, or use a self-balancing tree (next step).
- Treaps/skip lists: randomization achieves balance with high probability.

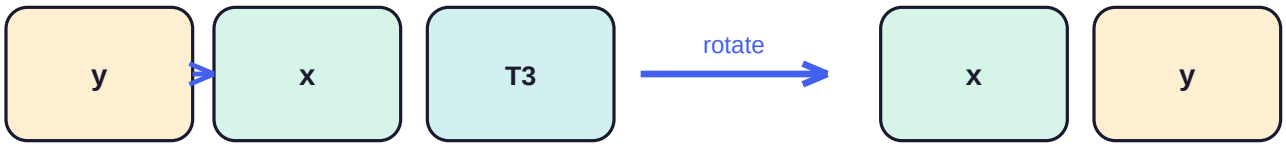
This is why 'just use a BST' is never the full answer — always ask how balance is maintained.

STEP 5/6

Self-Balancing: The Rotation Idea

Restore balance locally with rotations — O(log n) guaranteed.

Right rotation (left-left case):



- Rotation rewires 3 pointers, preserves inorder (sorted) order — O(1) local fix.
- AVL: height diff at most 1, rotate on violation — strictest balance, fastest lookup.
- Red-black: looser color rules, fewer rotations — the pragmatic default (Linux, Java TreeMap, C++ map).
- B-trees: the disk edition — wide nodes, minimal I/O. Databases live here.

You won't hand-roll these in production — but knowing WHY the stdlib map is O(log n) separates users from engineers.

STEP 6/6

Tree Height vs Performance

Every tree algorithm's cost is a function of height.

Balanced binary (n=1M)

height ~ 20 -> search O(log n)

Degenerate chain (n=1M)

height = 1M -> search O(n)

B-tree (n=1M, fanout 100)

height ~ 3 -> 3 disk reads

Trie (keys length L)

height = L -> lookup O(L)

Rule of thumb: if your tree can degenerate, it will — in production, on a Friday, with sorted input.

STEP ✓I✓

Key Takeaways

Trees — Remember

- Vocabulary: root, leaf, depth, height; n nodes -> n-1 edges, one path between nodes.
- BST invariant (smaller keys left, larger right) makes search/insert/delete O(h) — height decides.
- Inorder = sorted, preorder = structure, postorder = children-first; BFS = level order.
- Sorted input degenerates naive BSTs to O(n) chains — self-balance (AVL/red-black) or randomize.
- Rotations fix balance locally in O(1) while preserving order; B-trees extend the idea to disk.

Keep exploring

Pair this guide with the matching video walkthroughs and hands-on labs at interstitiumlabs.dev